

Retirement Module - Comprehensive Technical Report

Version: v0.4.1 **Last Updated:** December 21, 2025 **Module Status:** Production Ready

Table of Contents

- 1. [Executive Summary](#)
- 2. [Backend Architecture](#)
- 3. [Database Structure](#)
- 4. [Frontend Architecture](#)
- 5. [API Routes](#)
- 6. [Business Logic & Calculations](#)
- 7. [Dependencies & Integrations](#)
- 8. [Testing Infrastructure](#)
- 9. [Key Features & Capabilities](#)
- 10. [Constants & Defaults](#)
- 11. [Architecture Patterns](#)
- 12. [Security & Authorization](#)
- 13. [Performance Considerations](#)
- 14. [File Paths Reference](#)

1. Executive Summary

The Retirement Module is a comprehensive UK-focused pension planning system that manages three types of pensions (DC, DB, State), provides retirement readiness analysis, Monte Carlo projections, annual allowance compliance, and intelligent strategy recommendations. It integrates deeply with the Investment module for portfolio analysis and Monte Carlo simulations.

Key Statistics

Metric	Count
Backend Services	8
API Controllers	2
Vue Components	18
Vue Views	8
Database Tables	4
API Endpoints	20+
Vuex Actions	19
Vuex Getters	15

2. Backend Architecture

2.1 Agent Layer - RetirementAgent

File: `app/Agents/RetirementAgent.php`

The RetirementAgent orchestrates all retirement business logic with 1-hour caching.

Constructor Dependencies

```
public function __construct(
    private PensionProjector $projector,
    private ReadinessScorer $scorer,
    private AnnualAllowanceChecker $allowanceChecker,
    private MonteCarloSimulator $monteCarloSimulator,
    private PortfolioAnalyzer $portfolioAnalyzer,
    private AssetAllocationOptimizer $allocationOptimizer,
    private FeeAnalyzer $feeAnalyzer,
    private TaxEfficiencyCalculator $taxCalculator
)
```

Key Methods

Method	Purpose	Returns
<code>analyze()</code>	Main retirement analysis orchestration	Complete analysis package
<code>generateRecommendations()</code>	Priority-based recommendations	Array of recommendations
<code>buildScenarios()</code>	What-if scenario building (4 types)	Scenario comparison data
<code>analyzeDCPensionPortfolio()</code>	Portfolio optimization analysis	Portfolio metrics

Scenario Types

- 1. **Current** - Baseline projection with existing parameters
- 2. **Increased Contribution** - Higher monthly contributions
- 3. **Later Retirement** - Delayed retirement age
- 4. **Lower Target** - Reduced income target

2.2 Controllers

RetirementController

File: `app/Http/Controllers/Api/RetirementController.php`

Main API controller with 15+ endpoints.

Method	Route	Purpose
<code>index()</code>	GET /	Get all retirement data
<code>analyze()</code>	POST /analyze	Run comprehensive analysis
<code>recommendations()</code>	GET /recommendations	Get recommendations
<code>scenarios()</code>	POST /scenarios	Build what-if scenarios
<code>getProjections()</code>	GET /projections	Get Monte Carlo projections
<code>getStrategies()</code>	GET /strategies	Get retirement strategies
<code>calculateStrategyImpact()</code>	GET /strategies/impact	Calculate strategy impact with sliders
<code>checkAnnualAllowance()</code>	GET /annual-allowance/{taxYear}	Check annual allowance
<code>analyzeDCPensionPortfolio()</code>	GET /portfolio-analysis	Analyze DC pension portfolios
<code>storeDCPension()</code>	POST /pensions/dc	Create DC pension
<code>updateDCPension()</code>	PUT /pensions/dc/{id}	Update DC pension
<code>destroyDCPension()</code>	DELETE /pensions/dc/{id}	Delete DC pension
<code>storeDBPension()</code>	POST /pensions/db	Create DB pension
<code>updateDBPension()</code>	PUT /pensions/db/{id}	Update DB pension
<code>destroyDBPension()</code>	DELETE /pensions/db/{id}	Delete DB pension
<code>updateStatePension()</code>	POST /state-pension	Update state pension

DCPensionHoldingsController

File: `app/Http/Controllers/Api/Retirement/DCPensionHoldingsController.php`

Manages individual holdings within DC pension pots.

Method	Route	Purpose
<code>index()</code>	GET /{dcPensionId}/holdings	List holdings for DC pension
<code>store()</code>	POST /{dcPensionId}/holdings	Add new holding
<code>update()</code>	PUT /{dcPensionId}/holdings/{holdingId}	Modify holding
<code>destroy()</code>	DELETE /{dcPensionId}/holdings/{holdingId}	Remove holding
<code>bulkUpdate()</code>	POST /{dcPensionId}/holdings/bulk-update	Bulk update holdings

2.3 Services (8 Total)

PensionProjector

File: `app/Services/Retirement/PensionProjector.php`

Pension value projections using Future Value formula.

```
// FV Formula:  $FV = PV(1+r)^n + PMT * [((1+r)^n - 1) / r]$ 

public function projectDCPension(DCPension $pension, ?float $growthRate = null): float
public function projectDBPension(DBPension $pension): float
public function projectStatePension(StatePension $pension): float
public function projectTotalRetirementIncome(User $user): array
public function calculateIncomeReplacementRatio(User $user): float
```

Key Calculations:

- Default growth rate: 5% annually
- Accounts for platform fees in growth rate
- Income extraction at 4% withdrawal rate (default)
- Full state pension (2024/25): £11,502

ReadinessScorer

File: `app/Services/Retirement/ReadinessScorer.php`

Retirement readiness analysis and scoring.

```
public function calculateReadinessScore(User $user): int           // 0-100
public function categorizeReadiness(int $score): string           //
Excellent/Good/Fair/Critical
public function calculateIncomeGap(User $user): float             //
Shortfall/surplus
public function getReadinessColor(int $score): string             //
green/amber/red
public function analyzeReadiness(User $user): array               // Complete
analysis
```

Score Categories:

Score	Category	Color
90+	Excellent	Green
70-89	Good	Green
50-69	Fair	Amber
0-49	Critical	Red

AnnualAllowanceChecker

File: `app/Services/Retirement/AnnualAllowanceChecker.php`

Pension annual allowance compliance.

```
public function checkAnnualAllowance(User $user, string $taxYear): array
public function calculateTapering(float $adjustedIncome): float
public function getCarryForward(User $user): float
public function checkMPAA(User $user): bool
```

Key Rules:

- Standard allowance: £60,000
- Tapered allowance: £1 reduction per £2 over threshold
- Minimum tapered allowance: £10,000
- MPAA (Money Purchase Annual Allowance): £10,000
- Carry forward: From previous 3 years

RetirementProjectionService

File: `app/Services/Retirement/RetirementProjectionService.php`

Monte Carlo projections for retirement.

```
public function getProjections(User $user): array
public function projectPensionPot(User $user): array
public function projectIncomeDrawdown(User $user): array
public function projectTargetIncomeDrawdown(User $user, float
$targetIncome): array
```

Configuration:

- Monte Carlo iterations: 1,000
- Default retirement age: 65
- Sustainable withdrawal rate: 4.7%
- Inflation rate: 2%
- Target income: 75% of current net income
- End age assumption: 100

RetirementStrategyService

File: `app/Services/Retirement/RetirementStrategyService.php` (55KB+)

Strategy recommendation engine with priority-based strategies.

Strategy Priority Order:

1. **Employer Match** - Maximize free money
2. **Contribution Increase** - Additional contributions

3. **Retirement Age** - Work longer
4. **Income Target Reduction** - Reduce spending

```
public function getStrategies(User $user): array
public function calculateAffordability(User $user): float
public function checkEmployerMatchStrategies(User $user): array
public function checkContributionIncreaseStrategy(User $user): array
public function checkRetirementAgeStrategy(User $user): array
public function checkIncomeTargetStrategy(User $user): array
```

Target Probability: 95% on-track **Constraint Checking:** Affordability, annual allowance, guaranteed income

DecumulationPlanner

File: `app/Services/Retirement/DecumulationPlanner.php`

Retirement income planning.

```
public function calculateSustainableWithdrawalRate(User $user): array
public function compareAnnuityVsDrawdown(User $user): array
```

Features:

- Tests 3%, 4%, 5% withdrawal rates
- Annuity vs flexible drawdown comparison
- Annuity rates vary by age and spouse status
- Default drawdown rate: 4%

ContributionOptimizer

File: `app/Services/Retirement/ContributionOptimizer.php`

Contribution strategy optimization.

```
public function optimizeContributions(User $user): array
public function checkEmployerMatch(User $user): array
public function calculateRequiredContribution(User $user): float
public function analyzeTaxRelief(User $user): array
```

PensionPortfolioAnalyzer

File: `app/Services/Retirement/PensionPortfolioAnalyzer.php`

DC pension portfolio analysis.

```
public function analyze(User $user): array
public function analyzeFees(User $user): array
public function buildBreakdown(User $user): array
```

Metrics Calculated:

- Risk metrics (Alpha, Beta, Sharpe Ratio, Volatility, Max Drawdown, VaR)
- Asset allocation analysis
- Diversification scoring
- Fee analysis and potential savings
- Holdings breakdown by pension

2.4 Models

RetirementProfile

File: `app/Models/RetirementProfile.php`

```
Schema:
- current_age: int
- target_retirement_age: int
- current_annual_salary: decimal(15,2)
- target_retirement_income: decimal(15,2)
- essential_expenditure: decimal(10,2)
- lifestyle_expenditure: decimal(10,2)
- life_expectancy: int
- spouse_life_expectancy: int
- risk_tolerance: enum('cautious', 'balanced', 'adventurous')

Relationships:
- BelongsTo User
```

DCPension

File: `app/Models/DCPension.php`

```
Schema:
- scheme_name: varchar(255)
- scheme_type: enum('workplace', 'sipp', 'personal')
- provider: varchar(255)
- pension_type: enum('occupational', 'sipp', 'personal', 'stakeholder')
- member_number: varchar(255)
- current_fund_value: decimal(15,2)
- annual_salary: decimal(10,2)
- employee_contribution_percent: decimal(5,2)
- employer_contribution_percent: decimal(5,2)
- employer_matching_limit: decimal(5,2)
```

```
- monthly_contribution_amount: decimal(10,2)
- lump_sum_contribution: decimal(15,2)
- investment_strategy: varchar(255)
- platform_fee_percent: decimal(5,4)
- retirement_age: int
- expected_return_percent: decimal(5,2)
- projected_value_at_retirement: decimal(15,2)
- risk_preference: enum('low', 'lower_medium', 'medium', 'upper_medium',
'high')
- has_custom_risk: boolean

Relationships:
- BelongsTo User
- MorphMany Holdings (polymorphic)
```

DBPension

File: `app/Models/DBPension.php`

```
Schema:
- scheme_name: varchar(255)
- scheme_type: enum('final_salary', 'career_average', 'public_sector')
- accrued_annual_pension: decimal(15,2)
- pensionable_service_years: decimal(5,2)
- pensionable_salary: decimal(10,2)
- normal_retirement_age: int
- revaluation_method: varchar(255)
- spouse_pension_percent: decimal(5,2)
- lump_sum_entitlement: decimal(15,2)
- inflation_protection: enum('cpi', 'rpi', 'fixed', 'none')

Relationships:
- BelongsTo User
```

StatePension

File: `app/Models/StatePension.php`

```
Schema:
- ni_years_completed: int (default 0)
- ni_years_required: int (default 35)
- state_pension_forecast_annual: decimal(10,2)
- state_pension_age: int
- already_receiving: boolean (default false)
- ni_gaps: json
- gap_fill_cost: decimal(10,2)

Relationships:
- BelongsTo User
```


2.5 Request Classes

Directory: [app/Http/Requests/Retirement/](#)

Class	Purpose	Key Validations
StoreDCPensionRequest	DC pension creation	scheme_name, current_fund_value, contributions
StoreDBPensionRequest	DB pension creation	scheme_type, accrued_annual_pension
UpdateStatePensionRequest	State pension update	ni_years_completed, forecast
RetirementAnalysisRequest	Analysis parameters	growth_rate (0-15%), inflation_rate (0-10%)
ScenarioRequest	Scenario parameters	increased_contribution, later_retirement_age

3. Database Structure

3.1 Tables

retirement_profiles

```
CREATE TABLE `retirement_profiles` (  
  `id` bigint unsigned NOT NULL AUTO_INCREMENT,  
  `user_id` bigint unsigned NOT NULL,  
  `current_age` int NOT NULL,  
  `target_retirement_age` int NOT NULL,  
  `current_annual_salary` decimal(15,2) DEFAULT NULL,  
  `target_retirement_income` decimal(15,2) DEFAULT NULL,  
  `essential_expenditure` decimal(10,2) DEFAULT NULL,  
  `lifestyle_expenditure` decimal(10,2) DEFAULT NULL,  
  `life_expectancy` int DEFAULT NULL,  
  `spouse_life_expectancy` int DEFAULT NULL,  
  `risk_tolerance` enum('cautious','balanced','adventurous') DEFAULT  
'balanced',  
  `created_at` timestamp NULL DEFAULT NULL,  
  `updated_at` timestamp NULL DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  KEY `retirement_profiles_user_id_index` (`user_id`),  
  CONSTRAINT `retirement_profiles_user_id_foreign`  
    FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE  
);
```

dc_pensions

```

CREATE TABLE `dc_pensions` (
  `id` bigint unsigned NOT NULL AUTO_INCREMENT,
  `user_id` bigint unsigned NOT NULL,
  `scheme_name` varchar(255) DEFAULT NULL,
  `scheme_type` enum('workplace','sipp','personal') DEFAULT NULL,
  `provider` varchar(255) DEFAULT NULL,
  `pension_type` enum('occupational','sipp','personal','stakeholder')
  DEFAULT 'occupational',
  `member_number` varchar(255) DEFAULT NULL,
  `current_fund_value` decimal(15,2) NOT NULL DEFAULT '0.00',
  `annual_salary` decimal(10,2) DEFAULT NULL,
  `employee_contribution_percent` decimal(5,2) DEFAULT NULL,
  `employer_contribution_percent` decimal(5,2) DEFAULT NULL,
  `employer_matching_limit` decimal(5,2) DEFAULT NULL,
  `monthly_contribution_amount` decimal(10,2) DEFAULT NULL,
  `lump_sum_contribution` decimal(15,2) DEFAULT NULL,
  `investment_strategy` varchar(255) DEFAULT NULL,
  `platform_fee_percent` decimal(5,4) DEFAULT NULL,
  `retirement_age` int DEFAULT NULL,
  `expected_return_percent` decimal(5,2) DEFAULT NULL,
  `projected_value_at_retirement` decimal(15,2) DEFAULT NULL,
  `risk_preference`
  enum('low','lower_medium','medium','upper_medium','high') DEFAULT NULL,
  `has_custom_risk` tinyint(1) NOT NULL DEFAULT '0',
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `dc_pensions_user_id_index` (`user_id`),
  CONSTRAINT `dc_pensions_user_id_foreign`
  FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE
);

```

db_pensions

```

CREATE TABLE `db_pensions` (
  `id` bigint unsigned NOT NULL AUTO_INCREMENT,
  `user_id` bigint unsigned NOT NULL,
  `scheme_name` varchar(255) DEFAULT NULL,
  `scheme_type` enum('final_salary','career_average','public_sector') NOT
  NULL,
  `accrued_annual_pension` decimal(15,2) DEFAULT NULL,
  `pensionable_service_years` decimal(5,2) DEFAULT NULL,
  `pensionable_salary` decimal(10,2) DEFAULT NULL,
  `normal_retirement_age` int DEFAULT NULL,
  `revaluation_method` varchar(255) DEFAULT NULL,
  `spouse_pension_percent` decimal(5,2) DEFAULT NULL,
  `lump_sum_entitlement` decimal(15,2) DEFAULT NULL,
  `inflation_protection` enum('cpi','rpi','fixed','none') DEFAULT 'none',

```

```

`created_at` timestamp NULL DEFAULT NULL,
`updated_at` timestamp NULL DEFAULT NULL,
PRIMARY KEY (`id`),
KEY `db_pensions_user_id_index` (`user_id`),
CONSTRAINT `db_pensions_user_id_foreign`
  FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE
);

```

state_pensions

```

CREATE TABLE `state_pensions` (
  `id` bigint unsigned NOT NULL AUTO_INCREMENT,
  `user_id` bigint unsigned NOT NULL,
  `ni_years_completed` int NOT NULL DEFAULT '0',
  `ni_years_required` int NOT NULL DEFAULT '35',
  `state_pension_forecast_annual` decimal(10,2) DEFAULT NULL,
  `state_pension_age` int DEFAULT NULL,
  `already_receiving` tinyint(1) NOT NULL DEFAULT '0',
  `ni_gaps` json DEFAULT NULL,
  `gap_fill_cost` decimal(10,2) DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `state_pensions_user_id_index` (`user_id`),
  CONSTRAINT `state_pensions_user_id_foreign`
    FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE
);

```

3.2 Relationships Diagram

```

users (1)
├─ retirement_profiles (1:1)
├─ dc_pensions (1:N)
│   └─ holdings (1:N via polymorphic MorphMany)
├─ db_pensions (1:N)
└─ state_pensions (1:1 via updateOrCreate)

```

3.3 Design Notes

- All retirement tables FK to users.id with CASCADE delete
- DCPension has polymorphic relationship to Investment.Holding via holdable_id/holdable_type
- RetirementProfile is effectively 1:1 with User
- StatePension uses updateOrCreate pattern (1:1)
- No explicit carry-forward table (simplified implementation)
- NI gaps stored as JSON array in state_pensions

4. Frontend Architecture

4.1 Vue Views

Directory: [resources/js/views/Retirement/](#)

View	Purpose	Key Features
RetirementReadiness.vue	Main dashboard	Pensions overview grid, add/edit modals
Projections.vue	Future value projections	Monte Carlo chart, probability bands
Recommendations.vue	Recommendations list	Priority-based, action buttons
WhatIfScenarios.vue	Scenario builder	Parameter inputs, comparison charts
ContributionsAllowances.vue	Allowance tracking	Annual allowance, carry forward, MPAA
DecumulationPlanning.vue	Income planning	Withdrawal rates, annuity comparison
PortfolioAnalysis.vue	DC holdings analysis	Risk metrics, asset allocation, fees
PensionDetail.vue	Individual pension	Holdings management, edit/delete

4.2 Vue Components

Directory: [resources/js/components/Retirement/](#)

Form Components

Component	Purpose
UnifiedPensionForm.vue	Three-step form selector (DC/DB/State)
DCPensionForm.vue	DC pension form with contribution options
DBPensionForm.vue	DB pension form
StatePensionForm.vue	State pension form

Analysis Components

Component	Purpose
ReadinessGauge.vue	Circular readiness score display
RetirementOverviewCard.vue	Summary card component
AnnualAllowanceTracker.vue	Allowance status visualization

Chart Components

Component	Purpose
AccumulationChart.vue	Pension pot growth chart
IncomeProjectionChart.vue	Income projection visualization
IncomeDrawdownChart.vue	Drawdown scenario chart
PensionPotProjectionChart.vue	Future value projection with bands
TargetIncomeDrawdownChart.vue	Target income scenarios

Tab Components

Component	Purpose
FutureValueTab.vue	Monte Carlo projections display
StrategiesTab.vue	Retirement strategy recommendations
StrategyCard.vue (18.8KB)	Interactive strategy with slider

Interactive Components

Component	Purpose
DrawdownSimulator.vue	Interactive withdrawal simulator

4.3 Vuex Store Module

File: resources/js/store/modules/retirement.js

State (28 Properties)

```
{
  dcPensions: [],
  dbPensions: [],
  statePension: null,
  profile: null,
  analysis: null,
  recommendations: [],
  annualAllowance: null,
  scenarios: null,
  portfolioAnalysis: null,
  projections: null,
  strategies: null,
  strategyImpact: null,
  loading: false,
  projectionsLoading: false,
  strategiesLoading: false,
  error: null
}
```

Mutations (17)

- SET/UPDATE/REMOVE for pensions
- SET for analysis, recommendations, allowance, scenarios
- SET for projections, strategies, strategyImpact
- SET_LOADING variants for async operations

Actions (19)

Action	Purpose
fetchRetirementData	Load all retirement data
fetchRecommendations	Get recommendations
fetchProjections	Get Monte Carlo projections
fetchStrategies	Get retirement strategies
fetchPortfolioAnalysis	Get portfolio analysis
analyseRetirement	Run comprehensive analysis
calculateStrategyImpact	Calculate strategy impact with slider values
runScenario	Execute what-if scenario
createDCPension	Create DC pension
updateDCPension	Update DC pension
deleteDCPension	Delete DC pension
createDBPension	Create DB pension
updateDBPension	Update DB pension
deleteDBPension	Delete DB pension
updateStatePension	Update state pension
checkAnnualAllowance	Check annual allowance for tax year
createDCPensionHolding	Add holding to DC pension
updateDCPensionHolding	Update DC pension holding
deleteDCPensionHolding	Remove DC pension holding
bulkUpdateDCPensionHoldings	Bulk update holdings

Getters (15)

Getter	Returns
--------	---------

Getter	Returns
totalPensionWealth	Sum of all pension values
retirementReadinessScore	Readiness score (0-100)
projectedIncome	Projected retirement income
targetIncome	Target retirement income
incomeGap	Gap between projected and target
yearsToRetirement	Years until retirement age
hasIncomeSurplus	Boolean - surplus detected
hasIncomeGap	Boolean - shortfall detected
hasPortfolioData	Boolean - portfolio data available
portfolioTotalValue	Total DC pension value
portfolioRiskMetrics	Risk metrics from analysis
portfolioAssetAllocation	Asset allocation breakdown
portfolioDiversificationScore	Diversification score
portfolioFeeAnalysis	Fee analysis data
pensionsWithHoldings	Pensions that have holdings

Request Deduplication

Store tracks ongoing requests to prevent duplicate API calls for:

- Recommendations
- Annual allowance
- Portfolio analysis

4.4 Frontend Service

File: resources/js/services/retirementService.js

Method	Endpoint	Purpose
getRetirementData()	GET /retirement	All retirement data
analyzeRetirement()	POST /retirement/analyze	Run analysis
getRecommendations()	GET /retirement/recommendations	Get recommendations
runScenario()	POST /retirement/scenarios	Build scenarios
getProjections()	GET /retirement/projections	Monte Carlo projections

Method	Endpoint	Purpose
<code>getStrategies()</code>	GET /retirement/strategies	Get strategies
<code>calculateStrategyImpact()</code>	GET /retirement/strategies/impact	Strategy impact
<code>getAnnualAllowance()</code>	GET /retirement/annual-allowance/{taxYear}	Check allowance
<code>createDCPension()</code>	POST /retirement/pensions/dc	Create DC pension
<code>updateDCPension()</code>	PUT /retirement/pensions/dc/{id}	Update DC pension
<code>deleteDCPension()</code>	DELETE /retirement/pensions/dc/{id}	Delete DC pension
<code>createDBPension()</code>	POST /retirement/pensions/db	Create DB pension
<code>updateDBPension()</code>	PUT /retirement/pensions/db/{id}	Update DB pension
<code>deleteDBPension()</code>	DELETE /retirement/pensions/db/{id}	Delete DB pension
<code>updateStatePension()</code>	POST /retirement/state-pension	Update state pension

5. API Routes

Base Route: `/api/retirement` Middleware: `auth:sanctum`

5.1 Main Endpoints

GET	/	- Get all retirement data
GET	/projections	- Get Monte Carlo projections
POST	/analyze	- Run comprehensive analysis
GET	/recommendations	- Get recommendations
POST	/scenarios	- Build what-if scenarios
GET	/annual-allowance/{taxYear}	- Check annual allowance
GET	/strategies	- Get retirement strategies
GET	/strategies/impact	- Calculate strategy impact
GET	/portfolio-analysis	- Analyze all DC pension
portfolios		
GET	/portfolio-analysis/{dcPensionId}	- Analyze specific pension
portfolio		

5.2 DC Pension Routes

POST	/pensions/dc	- Create DC pension
PUT	/pensions/dc/{id}	- Update DC pension
DELETE	/pensions/dc/{id}	- Delete DC pension
GET	/pensions/dc/{id}/holdings	- List holdings
POST	/pensions/dc/{id}/holdings	- Create holding
PUT	/pensions/dc/{id}/holdings/{holdingId}	- Update holding

DELETE	/pensions/dc/{id}/holdings/{holdingId}	- Delete holding
POST	/pensions/dc/{id}/holdings/bulk-update	- Bulk update holdings

5.3 DB Pension Routes

POST	/pensions/db	- Create DB pension
PUT	/pensions/db/{id}	- Update DB pension
DELETE	/pensions/db/{id}	- Delete DB pension

5.4 State Pension Route

POST	/state-pension	- Update state pension
------	----------------	------------------------

6. Business Logic & Calculations

6.1 Retirement Readiness Score

Formula: $(\text{projected_income} / \text{target_income}) * 100$

Categories:

Score	Category	Color	Description
90+	Excellent	Green	On track for comfortable retirement
70-89	Good	Green	Minor adjustments may help
50-69	Fair	Amber	Action needed to meet goals
0-49	Critical	Red	Significant gap requires attention

6.2 DC Pension Projection

Future Value Formula:

$$FV = PV(1+r)^n + PMT * [((1+r)^n - 1) / r]$$

Where:

- PV = Current fund value
- r = (Growth rate - Platform fee) / 12 (monthly)
- n = Months to retirement
- PMT = Monthly contribution

Default Parameters:

- Growth rate: 5% annually
- Retirement age: 67
- Platform fee: Deducted from growth rate

6.3 Annual Allowance Calculation

Standard Allowance: £60,000

Tapered Allowance (High Earners):

```
If adjusted income > threshold income:  
Reduction = (adjusted income - threshold) / 2  
Tapered allowance = £60,000 - Reduction  
Minimum = £10,000
```

Carry Forward:

- Unused allowance from previous 3 tax years
- Simplified as 1 year's allowance in current implementation

MPAA (Money Purchase Annual Allowance):

- Triggered when flexible access taken
- Reduces allowance to £10,000

6.4 Strategy Prioritization

Order:

1. **Employer Match Optimization** (Maximize free money)

- Check workplace pension matching limits
- Identify underutilized employer matching

2. **Contribution Increase** (Additional saving)

- Calculate required contribution to meet target
- Check affordability (monthly disposable income)
- Check annual allowance remaining
- Suggest monthly amount needed

3. **Retirement Age Adjustment** (Work longer)

- Test higher retirement ages (up to 75)
- Calculate additional growth and contributions
- Additional years * 5% growth (simplified)

4. **Income Target Reduction** (Reduce spending)

- Adjust target income downward
- Calculate required reduction percentage

- Assess lifestyle impact

Target Probability: 95% on-track

6.5 Income Drawdown Analysis

Withdrawal Rates Tested: 3%, 4%, 5%

Default Drawdown Rate: 4% (4% rule)

Annuity Comparison:

- Annuity rates vary by age (5-6% for age 65-70)
- Adjusted for spouse pension
- Compares guaranteed vs. flexible income

6.6 State Pension Calculation

Full State Pension (2024/25): £11,502 per annum

Pro-rata Calculation:

$$\text{forecast} = (\text{ni_years_completed} / \text{ni_years_required}) * \text{full_amount}$$

Required NI Years: 35 years for full pension

6.7 Monte Carlo Simulation

Implementation: Box-Muller Transform for normal distribution

Parameters:

- Iterations: 1,000
- Growth rate: Risk-adjusted based on profile
- Volatility: Risk-adjusted based on profile

Output Percentiles:

- 10th percentile (pessimistic)
- 25th percentile
- 50th percentile (median)
- 75th percentile
- 90th percentile (optimistic)

7. Dependencies & Integrations

7.1 Internal Service Dependencies

Service	Source	Usage
---------	--------	-------

Service	Source	Usage
TaxConfigService	Core	Tax allowances and rates
UKTaxCalculator	Core	Tax calculations
UserProfileService	Profile	User income and profile data
RiskPreferenceService	Investment	Risk parameters for returns/volatility
MonteCarloSimulator	Investment	Probabilistic projections
PortfolioAnalyzer	Investment	Risk metrics, asset allocation
AssetAllocationOptimizer	Investment	Target allocation
FeeAnalyzer	Investment	Fee analysis
TaxEfficiencyCalculator	Investment	Tax optimization

7.2 Module Integrations

Investment Module:

- Uses Investment.Holding model for DC pension holdings (polymorphic)
- Leverages Investment risk profiles
- Shares MonteCarloSimulator, PortfolioAnalyzer, FeeAnalyzer
- Tax efficiency calculations shared

User Profile/Tax Module:

- Current annual salary
- Tax rates and allowances
- NI contributions

Estate Planning Module:

- Pension death benefits context
- Lifetime allowance implications

7.3 Polymorphic Holdings

DC Pension holdings use Laravel's polymorphic relationship:

```
// In DCPension model
public function holdings()
{
    return $this->morphMany(Holding::class, 'holdable');
}

// Holdings table
- holdable_id: bigint (DCPension ID)
- holdable_type: 'App\Models\DCPension'
```

8. Testing Infrastructure

8.1 Unit Tests (Backend)

Directory: `tests/Unit/Services/Retirement/`

Test File	Coverage
<code>AnnualAllowanceCheckerTest.php</code>	Allowance calculations, tapering
<code>DecumulationPlannerTest.php</code>	Withdrawal rates, annuity comparison
<code>PensionProjectorTest.php</code>	FV calculations, projections
<code>ReadinessScorerTest.php</code>	Score calculations, categorization

8.2 Feature Tests (Backend)

Directory: `tests/Feature/`

Test File	Coverage
<code>RetirementIntegrationTest.php</code>	Full module integration
<code>RetirementModuleTest.php</code>	API endpoint testing

8.3 Frontend Tests (Jest/Vue)

Directory: `tests/frontend/components/Retirement/`

Test File	Coverage
<code>AccumulationChart.test.js</code>	Chart rendering
<code>AnnualAllowanceTracker.test.js</code>	Tracker component
<code>DrawdownSimulator.test.js</code>	Simulator interaction
<code>IncomeProjectionChart.test.js</code>	Chart data handling
<code>PensionCard.test.js</code>	Card display
<code>ReadinessGauge.test.js</code>	Gauge rendering
<code>RetirementOverviewCard.test.js</code>	Overview display

9. Key Features & Capabilities

9.1 Pension Management

- **Multi-pension support:** Workplace, SIPP, Personal (DC); Final Salary, Career Average, Public Sector (DB)
- **Current value tracking** with fund value projections

- **Monthly/annual contribution tracking**
- **Employer matching analysis**
- **Platform fee considerations**
- **Risk preference per pension** (can override main profile)

9.2 Analysis & Projections

- **Monte Carlo simulations** (1,000 iterations) for probabilistic outcomes
- **Probability bands** for future value projections
- **Income replacement ratio** calculations
- **Retirement readiness scoring** (0-100)
- **Income gap analysis** (shortfall/surplus)

9.3 Recommendations

- **Priority-based strategy suggestions**
- **Automatic calculation** of required contributions
- **Employer match optimization**
- **Retirement age impact analysis**
- **Income target adjustment scenarios**

9.4 Tax Compliance

- **Annual allowance checking**
- **Tapered allowance calculations**
- **Carry forward** from previous 3 years
- **MPAA detection**
- **NI gap filling** for state pension

9.5 Interactive Features

- **What-if scenario builder**
- **Strategy impact sliders** with real-time calculations
- **Drawdown simulator**
- **Annuity vs drawdown comparison**
- **Portfolio holdings management** for DC pensions

9.6 Portfolio Optimization

- **Risk metrics:** Alpha, Beta, Sharpe Ratio, Volatility, Max Drawdown, VaR
- **Asset allocation analysis**
- **Diversification scoring**
- **Fee analysis** and potential savings
- **Holdings breakdown by pension**

10. Constants & Defaults

From RetirementProjectionService

```
DEFAULT_RETIREMENT_AGE = 65
SUSTAINABLE_WITHDRAWAL_RATE = 0.047 (4.7%)
INFLATION_RATE = 0.02 (2%)
TARGET_INCOME_PERCENT = 0.75 (75% of current net income)
END_AGE = 100
MONTE_CARLO_ITERATIONS = 1000
```

From DecumulationPlanner

```
TESTED_WITHDRAWAL_RATES = [0.03, 0.04, 0.05] (3%, 4%, 5%)
ANNUITY_RATES = [
    65 => 0.05, // 5%
    66 => 0.052,
    67 => 0.054,
    68 => 0.056,
    69 => 0.058,
    70 => 0.06 // 6%
]
```

From PensionProjector

```
DEFAULT_GROWTH_RATE = 0.05 (5%)
FULL_STATE_PENSION_2024_25 = 11502 (£11,502)
```

From RetirementStrategyService

```
ON_TRACK_PROBABILITY = 0.95 (95%)
```

Annual Allowance (2024/25)

```
STANDARD_ALLOWANCE = 60000 (£60,000)
MINIMUM_TAPERED = 10000 (£10,000)
MPAA = 10000 (£10,000)
```

11. Architecture Patterns

11.1 Agent Pattern

RetirementAgent orchestrates analysis across multiple services with caching.

```
public function analyze(int $userId): array
{
    return Cache::remember(
        "retirement_analysis_{$userId}",
        3600, // 1 hour
        fn() => $this->performAnalysis($userId)
    );
}
```

11.2 Service Pattern

8 specialized services each with single responsibility:

- PensionProjector → Projection calculations
- ReadinessScorer → Scoring and categorization
- AnnualAllowanceChecker → Tax compliance
- RetirementProjectionService → Monte Carlo
- RetirementStrategyService → Strategy recommendations
- DecumulationPlanner → Income planning
- ContributionOptimizer → Contribution strategy
- PensionPortfolioAnalyzer → Portfolio analysis

11.3 Polymorphic Model Pattern

Holdings can belong to DCPension or InvestmentAccount:

```
// Holding model
public function holdable()
{
    return $this->morphTo();
}
```

11.4 Strategy Pattern

Multiple calculation strategies with priority ordering:

```
$strategies = [
    $this->checkEmployerMatchStrategies($user),
    $this->checkContributionIncreaseStrategy($user),
    $this->checkRetirementAgeStrategy($user),
    $this->checkIncomeTargetStrategy($user),
];
```

11.5 Vue State Management

Namespaced Vuex module with request deduplication:


```
if (state.pendingRequests.recommendations) {  
    return state.pendingRequests.recommendations;  
}  
state.pendingRequests.recommendations = api.get('/recommendations');
```

12. Security & Authorization

12.1 Authentication

- All routes require `auth:sanctum` middleware
- Bearer token authentication

12.2 Authorization

- User ownership checks in all CRUD operations
- Authorization via request classes for updates
- Pension operations scoped to authenticated user only

12.3 Data Protection

- No hardcoded tax values (use `TaxConfigService`)
- Sensitive calculations protected in services
- Input validation on all endpoints

13. Performance Considerations

13.1 Caching

- 1-hour caching on analysis results
- Cache key: `retirement_analysis_{userId}`
- Cleared on pension CRUD operations

13.2 Request Deduplication

Frontend store tracks ongoing requests:

- Prevents duplicate API calls
- Returns existing promise if request in flight

13.3 Monte Carlo Optimization

- Limited to 1,000 iterations for balance of accuracy/speed
- Uses efficient Box-Muller transform
- Bulk percentile calculation

13.4 Database Optimization

- Indexed foreign keys on user_id
- Eager loading with `with()` to prevent N+1
- Minimal queries via relationship loading

14. File Paths Reference

Backend

```
app/Agents/  
└─ RetirementAgent.php  
  
app/Http/Controllers/Api/  
├─ RetirementController.php  
└─ Retirement/  
    └─ DCPensionHoldingsController.php  
  
app/Services/Retirement/  
├─ PensionProjector.php  
├─ ReadinessScorer.php  
├─ AnnualAllowanceChecker.php  
├─ RetirementProjectionService.php  
├─ RetirementStrategyService.php  
├─ DecumulationPlanner.php  
├─ ContributionOptimizer.php  
└─ PensionPortfolioAnalyzer.php  
  
app/Models/  
├─ RetirementProfile.php  
├─ DCPension.php  
├─ DBPension.php  
└─ StatePension.php  
  
app/Http/Requests/Retirement/  
├─ StoreDCPensionRequest.php  
├─ StoreDBPensionRequest.php  
├─ UpdateStatePensionRequest.php  
├─ RetirementAnalysisRequest.php  
└─ ScenarioRequest.php  
  
database/factories/  
└─ RetirementProfileFactory.php
```

Frontend

```
resources/js/views/Retirement/  
├─ RetirementReadiness.vue  
├─ Projections.vue  
├─ Recommendations.vue  
└─ WhatIfScenarios.vue
```

```
├── ContributionsAllowances.vue
├── DecumulationPlanning.vue
├── PortfolioAnalysis.vue
├── PensionDetail.vue

resources/js/components/Retirement/
├── UnifiedPensionForm.vue
├── DCPensionForm.vue
├── DBPensionForm.vue
├── StatePensionForm.vue
├── ReadinessGauge.vue
├── RetirementOverviewCard.vue
├── AccumulationChart.vue
├── IncomeProjectionChart.vue
├── IncomeDrawdownChart.vue
├── PensionPotProjectionChart.vue
├── TargetIncomeDrawdownChart.vue
├── FutureValueTab.vue
├── StrategiesTab.vue
├── StrategyCard.vue
├── AnnualAllowanceTracker.vue
├── DrawdownSimulator.vue

resources/js/store/modules/
├── retirement.js

resources/js/services/
├── retirementService.js
```

Routes

```
routes/api.php (lines 702–744)
```

Tests

```
tests/Unit/Services/Retirement/
├── AnnualAllowanceCheckerTest.php
├── DecumulationPlannerTest.php
├── PensionProjectorTest.php
├── ReadinessScorerTest.php

tests/Feature/
├── RetirementIntegrationTest.php
├── RetirementModuleTest.php

tests/frontend/components/Retirement/
├── AccumulationChart.test.js
├── AnnualAllowanceTracker.test.js
├── DrawdownSimulator.test.js
```

```
|— IncomeProjectionChart.test.js  
|— PensionCard.test.js  
|— ReadinessGauge.test.js  
|— RetirementOverviewCard.test.js
```

Conclusion

The Retirement Module provides comprehensive UK pension planning with:

- **Multi-pension support:** DC, DB, and State pension types
- **Advanced projections:** Monte Carlo simulations with probability bands
- **Tax compliance:** Annual allowance, tapering, carry forward, MPAA
- **Intelligent strategies:** Priority-based recommendations with real-time impact
- **Portfolio integration:** Shared holdings with Investment module
- **Interactive tools:** Scenario builders, simulators, strategy sliders

The module follows clean architecture principles with separation of concerns through the Agent/Service/Model layers, efficient caching strategies, and comprehensive test coverage.